GATE & PSUs Data Structures Question Bank

50 MCQS WITH EXAMINATION YEARS & COMPLETE SOLUTIONS

Stacks

Question 1 GATE 2016 / ISRO 2018

An element X is pushed onto a stack, then Y is pushed, then the stack is popped, and Z is pushed. What is the sequence of popped elements if we empty the stack?

A) X, Y, Z
B) Y, Z, X
C) Y, X, Z
D) Z, Y, X

Correct Answer: B) Y, Z, X

Explanation: Trace: Push(X) -> [X]; Push(Y) -> [X, Y]; Pop() returns Y, leaving [X]; Push(Z) -> [X, Z]; Emptying the stack pops Z, then X.

Question 2 GATE 2019 / BARC 2021

What is the postfix equivalent of the infix expression A + B * (C - D) / E, and what is the maximum size of the operator stack during conversion?

A) A B C D - * E / +, Max Size 3
B) A B C D - * E / +, Max Size 4
C) A B C * D - E / +, Max Size 4
D) A B C D - * / E +, Max Size 3

Correct Answer: B) A B C D - * E / +, Max Size 4

Explanation: The postfix expression evaluates to A B C D - * E / +. The operator stack grows to its maximum depth containing elements ['+', '(', '*', '-'] when scanning the token '-' inside the parentheses.

Question 3 GATE 2015 / DRDO 2014

What is the minimum number of stacks required to efficiently implement a FIFO queue structure where operations take amortized O(1) time?

A) 1
B) 2
C) 3
D) 4

Correct Answer: B) 2

Explanation: Two stacks are necessary: Stack1 tracks Enqueue, while Stack2 acts as the extraction frame for Dequeue operations. Elements are moved into Stack2 only when it is fully empty, which gives an amortized cost of O(1).

Question 4 GATE 2022 / CIL 2020

Evaluate the following postfix expression: 5 3 + 8 2 / * 4 -. What is the final value remaining on top of the execution stack?

A) 24
B) 28
C) 32
D) 16

Correct Answer: B) 28

Explanation: Trace: (5+3)=8. Then (8/2)=4. Next (8*4)=32. Finally, 32 - 4 = 28.

Question 5 GATE 2014 / NIELIT 2019

Let S be a stack of capacity n. If a sequence of n pushes and n pops are performed on an initially empty stack, the maximum number of unique valid output permutations that can be generated is given by:

A) n!
B) 2^n
C) The n-th Catalan Number
D) n^(n-1)

Correct Answer: C) The n-th Catalan Number

Explanation: The total valid combinations match the dynamic constraints of structural well-formed nested parentheses configurations, given mathematically by the Catalan number formula: [1/(n+1)] * C(2n, n).

Question 6 ISRO 2017 / Vizag Steel 2018

Which data structure is natively utilized by operating system runtimes to implement subprogram function execution calls and track local variable allocation memory blocks?

A) FIFO Queue

B) Singly Linked List

C) Activation Record Stack

D) Max-Heap

Correct Answer: C) Activation Record Stack

Explanation: A runtime stack frame isolates local parameters and tracks code memory return vectors sequentially as control moves in and out of active scopes.

Question 7 GATE 2020 / BARC 2022

To extend a standard stack implementation to support a constant time O(1) GetMin() tracking query operation without changing Push/Pop asymptotes, what is the optimal auxiliary spatial memory allocation needed?

A) O(1) extra space

B) O(log n) extra space

C) O(n) auxiliary space via an accompanying tracking stack

D) O(n^2) space matrix

Correct Answer: C) O(n) auxiliary space via an accompanying tracking stack

Explanation: An auxiliary parallel min-stack dynamically caches current local minimum variants mirroring the active stack size step-by-step.

Question 8 GATE 2017 / ISRO 2020

Determine the correct prefix transformation of the target postfix string sequence: A B + C * D -

A) - * + A B C D

B) - + * A B C D

C) * + - A B C D

D) - * A B + C D

Correct Answer: A) - * + A B C D

Explanation: Convert tokens by precedence: (A B +) becomes +AB. Next, (+AB) C * becomes *+ABC. Finally, (*+ABC) D - yields -*+ABCD.

Question 9 GATE 2013 / DRDO 2015

If numerical keys 1, 2, 3, 4 are sequentially submitted to a single stack allocation frame, which of the following output exit sequences cannot be generated?

A) 1, 2, 3, 4
B) 4, 3, 2, 1
C) 2, 3, 1, 4
D) 4, 1, 2, 3

Correct Answer: D) 4, 1, 2, 3

Explanation: To pull element 4 out first, 1, 2, and 3 must already reside inside the physical storage columns beneath it. Popping 4 exposes index 3 immediately; exiting 1 before 3 or 2 breaks stack laws.

Question 10 GATE 2011 / CIL 2021

A single linear array setup of size N needs to house two independent stacks. To achieve maximum space efficiency across variable dynamic loads, how should the storage spaces be configured?

A) Stack1 handles even locations, Stack2 handles odd locations
B) Stacks grow from opposite bounds of the array toward each other
C) Stack1 occupies indices 0 to N/2, Stack2 occupies the remainder
D) Interleave allocations dynamically based on time markers

Correct Answer: B) Stacks grow from opposite bounds of the array toward each other

Explanation: Configuring Stack1 from index 0 moving right and Stack2 from index N-1 moving left guarantees overflow occurs strictly when total elements across both equal array capacity N.

Question 11 ISRO 2015 / NIELIT 2021

What is the worst-case time complexity required to systematically reverse the elements of a stack of size n using only one single secondary auxiliary stack frame?

A) O(1)
B) O(n)
C) O(n log n)
D) O(n^2)

Correct Answer: D) O(n^2)

Explanation: Without dynamic pointer access or recursion arrays, moving elements iteratively layer-by-layer back and forth scales as a quadratic progression sequence.

Question 12 GATE 2018 / BARC 2019

When implementing a classic stack structure using a standard Singly Linked List, where should the Push and Pop code executions focus to achieve strict O(1) runtime bounds?

A) At the Tail node boundary

B) At the Head node boundary

C) Middle insertion point via fast pointer tracking

D) Dynamic alternating execution schedules

Correct Answer: B) At the Head node boundary

Explanation: The Head boundary supports O(1) insertion and isolation deletions. Tail updates run in O(n) line traversals unless complex bidirectional tracking states are kept.

Question 13 GATE 2021 / DRDO 2021

Consider an enhanced stack configuration supporting an aggregate macro command popK(k) which extracts k items simultaneously. What is the amortized cost per call across a continuous operation block?

A) O(1)

B) O(k)

C) O(n)

D) O(n log n)

Correct Answer: A) O(1)

Explanation: Though an isolated call might perform k work, any element popped could only be loaded once initially via an individual O(1) operation, yielding a constant amortized bound over time.

Question 14 GATE 2012 / ISRO 2014

Which standard Binary Search Tree tree node traversal pass strategy can be easily written without standard recursive engines by leveraging a single loop and stack template?

A) Level-order traversal

B) In-order traversal

C) Post-order lookup variation only

D) Non-directional search paths

Correct Answer: B) In-order traversal

Explanation: An explicit tracking stack enables the implementation to trace deep into left branches while recording parent nodes for later retrieval.

Question 15 GATE 2008 / CIL 2017

An arithmetic expression contains unbalanced parentheses. Describe a stack-based algorithm to find the first mismatched closing parenthesis.

A) Scan right-to-left, push closing brackets

B) Scan left-to-right, push open brackets, pop on close brackets; empty check triggers error

C) Count aggregate values through integer variables

D) Pass references into nested lookup indices

Correct Answer: B) Scan left-to-right, push open brackets, pop on close brackets; empty check triggers error

Explanation: When parsing left-to-right, an unmatched closing bracket causes a pop attempt on an empty stack, pointing exactly to the failure index.

Question 16 GATE 2023 / NIELIT 2022

For a mathematically valid postfix expression configuration consisting of exactly n distinct scalar operand components, what is the maximum possible tally of required binary operators?

A) n

B) n - 1

C) n + 1

D) 2n

Correct Answer: B) n - 1

Explanation: A binary operator joins two independent operand components down into a single result expression. Collapsing n elements down requires exactly n - 1 operators.

Question 17 GATE 2010 / BARC 2020

How many concurrent active stack frames are generated inside system execution scopes when tracking the calculation of the n-th Fibonacci value using standard unoptimized naive recursion paths?

A) O(n)

B) O(log n)

C) O(2^n)

D) O(n^2)

Correct Answer: C) O(2^n)

Explanation: Unoptimized Fibonacci calculations split into twin branch lines across every node step, generating an exponential execution call tree.

Queues

Question 18 GATE 2014 / ISRO 2019

A circular queue tracking layout utilizes an array setup of scale capacity N. If the 'front' index flags the start entry and 'rear' specifies the tail location, which calculation confirms a Full block state?

A) rear == front

B) (rear + 1) % N == front

C) rear + 1 == front

D) (front + 1) % N == rear

Correct Answer: B) (rear + 1) % N == front

Explanation: Reserving one empty slot keeps the Full state expression distinct from the Empty state representation (`front == rear`).

Question 19 GATE 2018 / DRDO 2017

If a system simulates a FIFO queue behavior using exactly two independent LIFO stack instances and designs Dequeue to carry the structural work overhead, what is the runtime of an Enqueue step?

A) O(1)

B) O(log n)

C) O(n)

D) O(n^2)

Correct Answer: A) O(1)

Explanation: When Dequeue is designed to handle element shifting, Enqueue operations are completed via a simple O(1) push directly onto the primary input stack.

Question 20 GATE 2017 / BARC 2017

In a circular queue tracking system containing elements indexed from 0 to M-1, which formula correctly computes the active structural size when given the current front and rear indices?

A) rear - front

B) (rear - front + M) % M

C) (front - rear + M) % M

D) rear + front % M

Correct Answer: B) (rear - front + M) % M

Explanation: Adding the total modulo base constant M handles wrap-around coordinates correctly when the rear index loops around behind the front index position.

Question 21 ISRO 2016 / CIL 2019

What is the structural characteristic defining a Deque (Double-Ended Queue) data structure compared to common queues?

A) It restricts deletions exclusively to internal nodes

B) Elements can be inserted or deleted from either end (front or rear)

C) It utilizes multidimensional tracking matrices

D) It forces a strict LIFO processing schedule

Correct Answer: B) Elements can be inserted or deleted from either end (front or rear)

Explanation: Deques generalize standard queues by supporting bidirectional entry and exit operations at both the front and rear boundaries.

Question 22 GATE 2015 / NIELIT 2018

Which fundamental graph discovery traversal strategy relies directly on the properties of a First-In-First-Out (FIFO) queue container?

A) Depth-First Search (DFS)

B) Topological Sorting Engine

C) Breadth-First Search (BFS)

D) Minimum Spanning Kruskal Path

Correct Answer: C) Breadth-First Search (BFS)

Explanation: BFS relies on a queue to evaluate graph nodes tier-by-tier in order of distance from the source.

Question 23 GATE 2012 / Vizag Steel 2021

If a queue is written utilizing a standard Singly Linked List, which configurations must be sustained to keep both Enqueue and Dequeue execution periods locked at O(1)?

A) Maintain a head pointer only

B) Maintain a tail pointer only

C) Maintain independent pointers to both the front (head) and rear (tail) nodes

D) Link entries into an orthogonal configuration grid

Correct Answer: C) Maintain independent pointers to both the front (head) and rear (tail) nodes

Explanation: A front pointer supports O(1) removals from the head, while a rear pointer enables O(1) additions to the tail.

Question 24 GATE 2019 / ISRO 2021

Consider a priority queue written as a binary Max-Heap layout. What are the time complexity boundaries for Insert and Extract-Max commands across an n-element block?

A) O(1) and O(n)

B) O(log n) and O(1)

C) O(log n) and O(log n)

D) O(n) and O(log n)

Correct Answer: C) O(log n) and O(log n)

Explanation: Both insertions and structural root removals require reheapify adjustments bounded by the overall depth of the binary tree layout.

Question 25 ISRO 2015 / BARC 2018

In multiprocess operating systems, how does a bounded circular queue clear roadblocks inside producer-consumer interaction tracks?

A) It randomizes thread processing schedules

B) It establishes a synchronized shared FIFO buffer that handles variable processing speeds

C) It converts async threads into static sequential codes

D) It drops excess data payloads when system loads peak

Correct Answer: B) It establishes a synchronized shared FIFO buffer that handles variable processing speeds

Explanation: The buffer decouples producer write speeds from consumer read operations, providing clean process safety boundaries.

Question 26 GATE 2011 / DRDO 2013

If a circular queue array implementation maintains an explicit helper integer variable called 'count' to track current load size, how many total slots of capacity N can be actively used?

A) N - 1 slots

B) N slots (all locations can be filled)

C) N / 2 slots

D) Zero slots

Correct Answer: B) N slots (all locations can be filled)

Explanation: Tracking full status via a counter variable allows the implementation to utilize all N slots without keeping a slot blank to avoid ambiguity.

Question 27 GATE 2021 / NIELIT 2020

Examine the following operation track sequence on an empty queue container structure: Enqueue(5), Enqueue(3), Dequeue(), Enqueue(2), Dequeue(). Which data component remains left inside?

A) [5]

B) [3]

C) [2]

D) [3, 2]

Correct Answer: C) [2]

Explanation: Trace: Enqueue(5)->[5]; Enqueue(3)->[5,3]; Dequeue() pops 5->[3]; Enqueue(2)->[3,2]; Dequeue() drops 3-> leaving [2].

Question 28 GATE 2016 / BARC 2015

Can a queue structure be implemented by using only a single physical stack?

A) No, stack physics cannot emulate FIFO

B) Yes, by utilizing the underlying system call stack recursively to handle item ordering

C) Yes, but only if stack size is limited to 2 slots

D) No, unless a hash map is attached

Correct Answer: B) Yes, by utilizing the underlying system call stack recursively to handle item ordering

Explanation: The recursive unwind process uses the system call stack frames to hold elements in memory, reversing them for FIFO extraction.

Question 29 ISRO 2014 / CIL 2018

Which scheduling template matches the standard operating system task layout design for managing Round-Robin CPU ready-states?

A) Priority Max-Heap Column

B) FIFO Queue structure

C) LIFO Stack column tracking

D) Disjoint-set forest mapping

Correct Answer: B) FIFO Queue structure

Explanation: Round-Robin scheduling processes tasks in a cycle, appending ready or timed-out processes to the rear of a FIFO queue.

Question 30 GATE 2007 / DRDO 2018

For an input-restricted deque tracking stream receiving data linear sequences labeled A, B, C, D, which output permutation sequence is impossible to construct?

A) A, B, C, D

B) D, C, B, A

C) C, D, B, A

D) D, C, A, B

Correct Answer: D) D, C, A, B

Explanation: Since inputs follow alphabetical order, A enters before B. If D and C are popped from the rear, A and B remain. A cannot exit before B from either end given the deque's constraints.

Question 31 ISRO 2023 / NIELIT 2023

What specific system error is triggered when software executes a Dequeue command targeting a queue instance that has zero active elements?

A) Buffer Overflow

B) Queue Underflow Error

C) Null Pointer Reference Exception

D) Index Allocation Limit Exceeded

Correct Answer: B) Queue Underflow Error

Explanation: Attempting to extract data from an empty container data structure triggers an underflow error.

Question 32 GATE 2020 / BARC 2023

What is the worst-case space complexity boundary of a Breadth-First Search (BFS) graph discovery run over a uniform tree with branching factor b and depth level d?

A) O(d)

B) O(b*d)

C) O(b^d)

D) O(1)

Correct Answer: C) O(b^d)

Explanation: BFS caches active nodes tier-by-tier; tracking the bottom leaf layer requires memory space proportional to the maximum tree width.

Question 33 GATE 2023 / ISRO 2022

How does a Monotonic Queue optimize runtime constraints when tracking evaluations across sliding window maximum analytics problems?

A) It changes inputs into randomized hash buckets

B) It prunes smaller and older elements to provide amortized O(1) update steps

C) It uses recursive matrix multipliers

D) It shifts structural configurations to secondary storage disks

Correct Answer: B) It prunes smaller and older elements to provide amortized O(1) update steps

Explanation: Maintaining elements in sorted order within the window allows the algorithm to find boundary maximums in overall linear time ($O(n)$).

Linked Lists

Question 34 GATE 2012 / ISRO 2017

Which tracking pointer constraint identifies a loop within a singly linked chain structure when executing Floyd's Cycle Finding Algorithm?

A) fast == NULL

B) slow == fast (pointers meet inside the loop)

C) slow == NULL

D) fast->next == NULL

Correct Answer: B) slow == fast (pointers meet inside the loop)

Explanation: The slow pointer advances by 1 step while the fast pointer advances by 2 steps. If a cycle exists, the fast pointer will close the distance and meet the slow pointer from behind.

Question 35 GATE 2010 / DRDO 2014

Given a pointer directly to an internal target node within a Singly Linked List (without access to the head pointer), what is the time complexity to delete that node?

A) O(1)

B) O(log n)

C) O(n)

D) O(n^2)

Correct Answer: A) O(1)

Explanation: Copy the data value from the next node into the target node, then bypass and delete that next node (`node->next = node->next->next`).

Question 36 GATE 2017 / BARC 2019

What is the optimal time complexity and auxiliary space required to merge two pre-sorted linked list chains of lengths M and N into a single combined sorted chain?

A) O(M+N) time and O(M+N) space

B) O(M log N) time and O(1) space

C) O(M+N) time and O(1) inline auxiliary space

D) O(M*N) time and O(1) space

Correct Answer: C) O(M+N) time and O(1) inline auxiliary space

Explanation: The lists can be merged in linear time by adjusting existing pointer links inline, avoiding new memory allocations.

Question 37 ISRO 2018 / CIL 2021

What structural advantage does a Doubly Linked List offer over common Singly Linked Lists?

A) It cuts pointer storage properties by half

B) It supports bidirectional traversal and true O(1) node deletions

C) It maps entries onto static contiguous memory slots

D) It eliminates head tracking configurations entirely

Correct Answer: B) It supports bidirectional traversal and true O(1) node deletions

Explanation: Each node maintains explicit references to both its predecessor and successor, enabling bidirectional traversal and direct deletions.

Question 38 GATE 2021 / NIELIT 2021

What is the most efficient method to locate the exact midpoint node of a singly linked chain layout during a single traversal pass?

A) Step through every element, then loop back through half the count

B) Deploy twin pointers, advancing the fast pointer at twice the speed of the slow pointer

C) Copy elements into a temporary array

D) Read the index data properties from a head metadata tag

Correct Answer: B) Deploy twin pointers, advancing the fast pointer at twice the speed of the slow pointer

Explanation: When the fast pointer reaches the end of the list, the slow pointer will be positioned exactly at the midpoint node.

Question 39 GATE 2016 / BARC 2021

How can a new node entry be inserted at the beginning of a Circular Singly Linked List in O(1) time without traversing the entire list to update the tail pointer?

A) It is impossible without a tail pointer reference

B) Insert the node after the head position and swap their data payloads

C) Temporarily break the circular chain reference

D) Allocate an auxiliary lookup hash frame

Correct Answer: B) Insert the node after the head position and swap their data payloads

Explanation: Inserting the new entry after the head node and swapping their data values avoids a full traversal to update the tail link.

Question 40 GATE 2015 / DRDO 2015

Compare the runtime and auxiliary memory footprints of reversing a singly linked chain iteratively versus recursively.

A) Iterative: O(n) time, O(n) space; Recursive: O(n) time, O(1) space

B) Iterative: O(n) time, O(1) space; Recursive: O(n) time, O(n) system call stack space

C) Iterative: O(n^2) time, O(1) space; Recursive: O(n) time, O(n) space

D) Both require matching O(n) time and O(1) space constraints

Correct Answer: B) Iterative: O(n) time, O(1) space; Recursive: O(n) time, O(n) system call stack space

Explanation: The iterative method updates pointers inline using a constant footprint, while recursion allocates call stack frames proportional to list length.

Question 41 GATE 2019 / ISRO 2020

Why do array-based list structures often outperform linked choices across sequential iteration tasks despite having identical asymptotic time bounds?

A) Arrays use less complex lookup algorithms

B) Contiguous array layouts maximize spatial locality of reference for higher cache hit rates

C) Linked structures incur runtime overhead due to bit-shift operations

D) Arrays execute operations using parallel processing hardware paths

Correct Answer: B) Contiguous array layouts maximize spatial locality of reference for higher cache hit rates

Explanation: Contiguous memory allows CPU caches to pre-load adjacent array elements, whereas scattered linked nodes trigger frequent, slower main memory fetches.

Question 42 ISRO 2014 / Vizag Steel 2019

What is the maximum number of structural comparisons required to sort a singly linked chain of n items using Bubble Sort logic?

A) O(n)

B) O(n log n)

C) O(n^2)

D) O(2^n)

Correct Answer: C) O(n^2)

Explanation: Bubble sort iterates through adjacent list nodes via nested loops, which scales quadratically ($n(n-1)/2$) in the worst case.

Question 43 GATE 2014 / BARC 2018

What structural feature allows Skip Lists to achieve O(log n) average search, insertion, and deletion performance boundaries?

A) They convert data values into balance codes

B) They maintain layered forward express lanes over sorted linked lists

C) They use multi-threaded execution paths

D) They compress pointer fields using XOR operations

Correct Answer: B) They maintain layered forward express lanes over sorted linked lists

Explanation: Layered express lanes skip segments of nodes, allowing search operations to skip forward and prune the remaining search space by half at each step.

Question 44 GATE 2013 / NIELIT 2019

To verify if a singly linked chain forms a palindrome structure by using a stack helper, what is the space complexity constraint?

A) O(1)

B) O(log n)

C) O(n)

D) O(n^2)

Correct Answer: C) O(n)

Explanation: Verifying a palindrome via a stack requires caching half the list nodes to validate them against the remaining elements, consuming linear space.

Question 45 GATE 2011 / DRDO 2012

How does an XOR Linked List store bidirectional link references while using only a single pointer field ('npx') per node?

A) It compresses bits using compression dictionaries

B) npx = pointer(prev) XOR pointer(next)

C) It multiplexes references based on memory clock cycles

D) It stores addresses in a centralized index grid

Correct Answer: B) npx = pointer(prev) XOR pointer(next)

Explanation: XORing the previous and next node addresses compresses bidirectional links into a single pointer field. Traversing requires deriving the next link via `next = npx XOR prev`.

Question 46 GATE 2008 / CIL 2017

When representing algebraic polynomials using linked chains, what do individual nodes typically capture?

A) Dynamic evaluation memory addresses

B) Term coefficients and exponent values

C) Direct variable hash codes

D) Matrix column indexes

Correct Answer: B) Term coefficients and exponent values

Explanation: Each node records a term's coefficient and exponent, sorted by exponent value to simplify operations like polynomial addition.

Question 47 ISRO 2015 / BARC 2020

What is the primary drawback of using singly linked chains to store random-access file index indices?

A) High pointer data overhead

B) Lack of constant-time O(1) random-access capabilities

C) Vulnerability to memory leak corruption

D) High operational bitwise complexity constraints

Correct Answer: B) Lack of constant-time O(1) random-access capabilities

Explanation: Finding an element at index k requires iterating through all preceding nodes sequentially from the head, resulting in poor search times ($O(n)$).

Question 48 GATE 2023 / NIELIT 2022

Given two singly linked list lines that intersect at an unknown node location, how can the intersection node be found with O(1) auxiliary space?

A) Use a nested lookup loop over both structures

B) Calculate lengths, advance the longer list's pointer by the difference, then step both pointers forward at equal speed

C) Track nodes using an auxiliary hash map container

D) Reverse both lists and check the head components

Correct Answer: B) Calculate lengths, advance the longer list's pointer by the difference, then step both pointers forward at equal speed

Explanation: Aligning the start positions by the length difference allows both pointers to advance together and meet exactly at the intersection node.

Question 49 GATE 2009 / DRDO 2019

What is the minimum number of link pointer fields needed per node to construct a multi-dimensional orthogonal linked list?

A) 1 field

B) 2 fields

C) 3 fields

D) 4 fields

Correct Answer: B) 2 fields

Explanation: An orthogonal grid requires at least two directional link fields (typically `Right` and `Down`) to maintain its grid structure.

Question 50 GATE 2020 / ISRO 2023

Why is MergeSort generally preferred over QuickSort for sorting linked list data structures?

A) MergeSort requires random-access indexing paths

B) MergeSort can split lists in O(n) time and perform merges using O(1) auxiliary space by adjusting pointers inline

C) QuickSort exhibits high spatial cache locality when processing node links

D) MergeSort uses fewer comparison steps

Correct Answer: B) MergeSort can split lists in O(n) time and perform merges using O(1) auxiliary space by adjusting pointers inline

Explanation: MergeSort handles sequential data efficiently because it can merge lists by adjusting pointer references inline without requiring the random access indexing that QuickSort relies on.